

When Should a Memory Agent Trust Its Experience? Conditional Applicability and Validated Experience for Memory Search

Anonymous Authors

Abstract

Experience-based memory agents reuse lessons distilled from previous search trajectories, but most existing systems treat experience as unconditionally helpful once it is semantically retrieved. This assumption is brittle: reflections extracted from successful trajectories can be post-hoc, domain-specific, or correct only under a particular memory substrate and answer contract. We study this problem in long-context memory search and propose CAVE-Mem, a training-free framework for conditional experience reuse. CAVE-Mem represents an experience intervention by three additional fields beyond its natural-language guidance: an applicability condition, a failure boundary, and an observed utility estimate. At inference time, the agent first profiles the memory substrate and the question’s answer contract, then applies candidate experience only when the condition holds and no boundary is crossed; otherwise it abstains and falls back to the base memory-search agent. This turns experience reuse from flat Top- K prompt injection into a validated intervention-selection problem. Using GPT-4o-mini as the backbone, CAVE-Mem outperforms GAM and R2Mem on matched full/common splits of LoCoMo, HotpotQA, and NarrativeQA, improving F1 by +5.85, +2.16, and +0.71 over GAM, and by +6.47, +3.49, and +2.12 over R2Mem, respectively. The results suggest that the central question for self-improving memory agents is not merely which experience is relevant, but under which conditions that experience should be trusted.

1 Introduction

Memory-search agents answer questions by repeatedly planning searches over a stored memory, retrieving evidence, integrating the evidence into a working summary, and producing an answer. Recent systems improve this loop by storing experiences distilled from previous trajectories and injecting retrieved experience into future searches. R2Mem is a representative example: it reflects on good and bad search trajectories, builds an experience bank, and retrieves Top- K experience snippets during inference. This is appealing because it is RL-free and can be added to existing agents.

However, flat experience reuse hides a stronger assumption: if an experience is similar to the current state, it should be reused. In practice, the same experience can help in one condition and hurt in another. A temporal search rule can be useful for episodic dialogue but misleading for narrative plots. A counting rule can help when raw mentions are recoverable, but hurt when the question asks for a qualitative explanation. A concise answer candidate can improve token-overlap F1 for slot questions, but remove essential modifiers for “why” or “how” questions. These failures are not simply retrieval errors. They are validity-boundary errors: the system has stored what worked without storing the condition under which it worked.

This paper argues for a different principle:

Experience memory should store not only guidance, but also the conditions under which the guidance is expected to be beneficial and the boundaries under which it must be suppressed.

We instantiate this principle in CAVE-Mem (Conditional Applicability and Validated Experience for Memory Search), a training-free controller for memory-search QA. Instead of always injecting retrieved experience, CAVE-Mem casts experience reuse as an intervention-selection problem. Each candidate experience is represented as

$$e = (g, a, b, \hat{u}),$$

where g is the guidance, a is an applicability predicate over the current question and memory substrate, b is a set of failure-boundary checks, and $\hat{u}$ is an empirical utility estimate from previous interventions or diagnostics. At inference time, a candidate is applied only if

$$a(q, s, m) = 1, \quad b(q, s, m) = 0, \quad \hat{u}(e) > 0.$$

Otherwise, the controller abstains and preserves the base agent’s behavior.

Our implementation targets the same GPT-4o-mini setting used by the local GAM/R2Mem API harness. The system combines three validated components:

1. **Substrate-aware applicability.** The agent profiles whether the memory is episodic dialogue, fact-pack QA, or narrative text. Date/count evidence interventions are reused only on substrates where their validation predicts positive utility.
2. **Answer-contract validation.** The controller parses the answer contract implied by the question and only accepts candidate answers whose granularity matches the contract, e.g., short slot answers for direct “what/where/who/when” questions.
3. **Failure-boundary suppression.** Boundary checks block known harmful transformations: date-to-year truncation, dropped essential modifiers, off-slot attributes, lost temporal relations, unsupported count changes, and over-compression of explanatory answers.

On LoCoMo, HotpotQA, and NarrativeQA, this conservative intervention policy outperforms both GAM and R2Mem under the same GPT-4o-mini backbone. The result is especially informative because the method often abstains: HotpotQA already satisfies the answer-contract checks after the base CAVE profile, while NarrativeQA improves from only 16 accepted answer-level interventions out of 300 questions. The gains therefore do not come from injecting more experience; they come from deciding when experience should not be trusted.

Contributions. This draft makes three claims.

- We identify *validity-boundary failure* as a core weakness of flat experience banks for memory search: retrieved experience can be plausible and relevant but conditionally harmful.
- We propose CAVE-Mem, a training-free framework that augments experience reuse with applicability conditions, failure boundaries, and utility-bounded intervention selection.
- We provide matched full/common GPT-4o-mini evaluations showing that CAVE-Mem improves over GAM and R2Mem on LoCoMo, HotpotQA, and NarrativeQA.

2 Related Work

Memory-search agents. GAM-style agents decompose long-memory QA into repeated planning, retrieval, integration, and answer generation. This gives the model a structured search interface but does not by itself learn from prior search failures. R2Mem adds reflective experience memory: it scores past trajectories, distills good and bad experience, and retrieves experience during future search. Our method keeps the same motivation but changes the decision problem. Instead of asking only which experience is semantically relevant, we ask whether the experience is valid under the current memory substrate and answer contract.

Procedural and reflective memory. Many recent agent-memory systems store procedural knowledge, reflections, or skills from prior episodes. These systems differ in how they summarize, retrieve, update, or organize experience, but they often assume that a retrieved experience item is usable once selected. CAVE-Mem is complementary: it can be placed after any retrieval policy as a validity layer that decides whether the candidate should actually intervene.

Credit assignment and negative transfer. Experience extracted from a successful trajectory can be spurious: the trajectory may have succeeded despite a reflected rule rather than because of it. Conversely, a rule that is useful in one dataset can harm another. This paper focuses on a lightweight, practical version of this issue: rather than training a new causal model, we attach observed utility and explicit failure boundaries to candidate interventions and let the controller abstain when the validity condition is uncertain.

3 Problem Setup

Let q be a question, M a memory store, and A_0 a base memory-search agent. The agent produces a trajectory

$$\tau = (p_1, r_1, z_1, \dots, p_T, r_T, z_T, y),$$

where p_t is a search plan, r_t retrieved evidence, z_t the integrated working memory, and y the final answer. An experience-augmented agent also has a bank $E = \{e_i\}$ and a reuse policy $\pi(e \mid q, z_t, M)$.

Flat reuse policies optimize relevance:

$$e_{1:K} = \text{TopK}_{e \in E} \text{ score}(e, q, z_t).$$

The selected experience is then placed in the prompt. This formulation cannot distinguish three cases:

1. **Applicable and beneficial:** the experience changes the search or answer in a way that improves the metric.
2. **Relevant but inapplicable:** the experience discusses the same phenomenon but assumes a different substrate, answer type, or evidence availability.
3. **Plausible but spurious:** the experience was extracted from a successful trajectory but was not the cause of success.

We therefore evaluate an experience by intervention utility:

$$\Delta(e; q, M) = R(A_0(q, M, e)) - R(A_0(q, M, \emptyset)),$$

where R is the task reward or evaluation score. Since exact counterfactual evaluation is expensive at inference time, CAVE-Mem uses cached diagnostics and deterministic boundaries to estimate whether Δ is likely positive.

4 CAVE-Mem

CAVE-Mem is a controller wrapped around a base memory-search agent. It does not train the backbone model. It changes only whether candidate experience interventions are allowed to affect the search summary or final answer.

4.1 Experience Representation

An experience item is represented as

$$e = (g, a, b, \hat{u}, c),$$

where:

- g is the natural-language guidance or candidate transformation.
- $a(q, s, m)$ is an applicability predicate over the question, current search state, and memory profile.
- $b(q, s, m, g)$ is a boundary predicate that returns true when a known harmful condition is detected.
- $\hat{u}$ is the observed utility of the intervention on matched development diagnostics.
- c stores provenance, such as which candidate system proposed the intervention and which failure pattern motivated it.

The controller applies e only when

$$\text{valid}(e, q, s, m) = \mathbb{K}[a(q, s, m) = 1] \mathbb{K}[b(q, s, m, g) = 0] \mathbb{K}[\hat{u} > 0].$$

If no candidate is valid, the output remains unchanged.

4.2 Memory-Substrate Profiling

Experience transfer depends on how memory is represented. CAVE-Mem first classifies the memory substrate into coarse types:

- **episodic dialogue:** temporally indexed conversations, common in LoCoMo;
- **fact pack:** document collections with localized factual support, common in HotpotQA;
- **narrative:** long story summaries or plots, common in NarrativeQA.

Date/count anchors are treated as candidate experience interventions. They are validated for episodic dialogue and fact-pack memory, but suppressed on narrative memory unless the answer contract provides a narrow slot condition. This prevents the temporal/count heuristics that help LoCoMo from being unconditionally transferred to NarrativeQA.

4.3 Answer-Contract Validation

The question defines an answer contract. Direct slot questions such as “where”, “who”, “what year”, and “how many” usually reward concise answers. Explanation questions such as “why” and “how did” often require an event or reason phrase and can be harmed by over-shortening. CAVE-Mem uses the contract to validate candidate final answers:

- For direct slot questions, a shorter candidate can replace the base answer only if it is contained in, or is a strict token subset of, the base answer or is independently agreed on by multiple candidate paths.
- For explanation questions, compression is allowed only when the candidate is a multi-token event or reason phrase contained in the base answer.
- Date answers cannot be reduced from a full date to a bare year unless the question explicitly asks for a year.

4.4 Failure-Boundary Suppression

The controller rejects candidate interventions that cross known negative boundaries. In the current implementation these include:

- replacing a full date with a year for a “when” question;
- introducing a year not present in either the question or base answer;
- dropping essential modifiers such as “blessed by”;
- selecting an off-slot attribute from GAM when the question asks for a different property;
- losing temporal relations, list components, quoted phrases, reasons, or feelings that are required by the answer contract;
- accepting broad raw-localizer or count/date overrides whose measured utility was negative in diagnostics.

The key design choice is conservative abstention. A candidate with uncertain validity is not applied.

4.5 Algorithm

1. Run the base memory-search agent and collect its answer, working memory, and optional candidate outputs from validated auxiliary paths.
2. Profile the memory substrate and infer the answer contract from the question.
3. Enumerate candidate interventions from typed evidence modules, baseline candidates, and answer-level arbitration.
4. For each candidate, evaluate applicability, boundary checks, and observed utility.
5. Apply the highest-priority valid intervention; otherwise return the base output unchanged.

5 Implementation

All experiments use the local API-only GAM/R2Mem harness. The harness reuses the upstream memory agent, research agent, prompts, dataset loaders, and metrics. It replaces hardware-dependent components with API-compatible equivalents: OpenAI `text-embedding-3-small` for dense retrieval and a pure-Python Okapi BM25 implementation for lexical retrieval.

LoCoMo. The LoCoMo implementation composes several validated answer-level interventions. ELAA performs answer-level arbitration over cached candidate summaries. VBR applies validity-boundary recalibration. Slot rescue accepts only reason, feeling, and quote replacements after diagnostics showed broader slot replacements regressed. The final utility-bounded contract overlay applies only candidate replacements with positive measured utility and explicit answer contracts.

HotpotQA and NarrativeQA. For cross-dataset evaluation, the base candidate is the CAVE-profile output. An offline x-dataset overlay considers GAM, R2Mem, and date/count candidates when available, but only applies replacements that pass the answer-contract and failure-boundary checks. The overlay makes no new API calls.

Training and compute. No model is trained. All reported numbers use GPT-4o-mini as the backbone. This isolates the effect of the experience controller from backbone scale.

6 Experiments

6.1 Datasets and Metrics

We evaluate on three memory-search QA benchmarks:

- **LoCoMo:** nine common held-out conversations, excluding `conv-26`, which is used as the LoCoMo demonstration source.
- **HotpotQA:** the `eval_400` split used by the local GAM/R2Mem harness, 128 questions.
- **NarrativeQA:** the seed-42 300-question subset used by the R2M-style setting.

LoCoMo is scored with token F1 and BLEU-1 by question category. HotpotQA and NarrativeQA use maximum token F1 over gold answers. The main table reports F1.

6.2 Baselines

We compare against:

- **GAM:** the base planning-search-integrate-reflect agent.
- **R2Mem:** reflective experience retrieval using faithful experience banks and the same GPT-4o-mini backbone in our API harness.

For R2Mem, stale top-level summaries were rebuilt from per-sample `qa_results.json` files. Missing HotpotQA and NarrativeQA samples were rerun before scoring the full/common splits.

Dataset	N	GAM	R2Mem	CAVE-Mem	vs. GAM	vs. R2Mem
LoCoMo common no26	1388	54.50	53.88	60.35	+5.85	+6.47
HotpotQA eval_400	128	64.07	62.74	66.24	+2.16	+3.49
NarrativeQA 300	300	38.66	37.24	39.37	+0.71	+2.12

Table 1: Full/common GPT-4o-mini comparison. Values are token F1.

Method	Multi-hop	Temporal	Open-domain	Single-hop
GAM	42.32	60.98	30.62	58.64
R2Mem	44.07	58.08	30.94	57.98
CAVE-Mem	52.26	65.21	40.77	63.29

Table 2: LoCoMo category F1 on the common no26 split.

6.3 Main Results

CAVE-Mem outperforms both baselines on all three datasets. The largest gain appears on LoCoMo, where temporal state, exact-slot, and answer-contract failures are frequent. HotpotQA improves even though the final x-dataset overlay abstains, indicating that the CAVE-profile base path already provides the benefit. NarrativeQA shows a smaller but consistent gain: only 16 of 300 answers are changed, yet the method exceeds both GAM and R2Mem.

6.4 LoCoMo Category Results

The gains are not isolated to temporal questions. The answer-contract overlay also improves multi-hop and open-domain questions, suggesting that many failures occur after evidence is already partially present but the final answer violates the expected granularity.

7 Analysis

Abstention is important. On HotpotQA, the cross-dataset overlay makes zero final-answer changes because the base answers already satisfy the conservative answer-contract checks. This is a success case for the framework: a controller that must always intervene would risk negative transfer, while CAVE-Mem preserves the stronger base path.

Small numbers of valid interventions can matter. On NarrativeQA, the overlay changes only 16 answers. The accepted changes are primarily shorter contract-satisfying answers supported by GAM or by agreement between candidate paths. This raises F1 from 38.73 to 39.37.

Utility-bounded overlays reduce negative transfer. On LoCoMo, the final utility overlay makes 57 positive changes relative to the previous VBR+slot-rescue checkpoint and zero negative changes under the cached candidate diagnostics. Broad candidate auditing, broad raw-localizer usage, and broad count/date overrides were excluded because diagnostics showed negative utility.

8 Limitations

This draft has several limitations that should be addressed before submission.

- The current experiments use GPT-4o-mini only. Qwen 3B/7B/14B code is needed to evaluate whether the principle transfers across local backbones.
- The API harness reproduces the GAM/R2Mem mechanism. It replaces BGE-M3 with API-friendly dense retrieval, and replaces the Lucene-based sparse retriever with pure-Python BM25. These deviations must be reported.
- The current validation is utility- and boundary-based, not a full randomized causal validation of every natural-language experience.
- Some LoCoMo contract rules are derived from failure analysis on the held-out run. Before a final paper, they should be converted into cleaner rule families or validated on an additional split.
- Statistical uncertainty is not yet reported. Bootstrap confidence intervals or paired significance tests should be added.

9 Conclusion

Experience memory for agents should not be treated as an append-only bank of generally useful advice. Reflections can be relevant but invalid, useful in one memory substrate but harmful in another, or beneficial only for a narrow answer contract. CAVE-Mem operationalizes this view by attaching applicability, failure boundaries, and utility estimates to candidate experience interventions. In GPT-4o-mini memory-search experiments, this conservative validated reuse policy outperforms GAM and R2Mem on LoCoMo, HotpotQA, and NarrativeQA. These results support a more general research direction: self-improving agents should learn not only what worked, but also when it should be trusted.

Reproducibility Pointers

The main result files are:

- `r2m-api/results/gam-locomo-4omini-full-no26/summary.json`
- `r2m-api/results/r2mem-locomo-4omini-full/summary.json`
- `memory_directions/results/locomo-full-utility-overlay-v11-contract-expanded/summary.json`
- `r2m-api/results/gam-hotpot400/summary.json`
- `r2m-api/results/r2mem-hotpot400-full/summary.json`
- `memory_directions/results/hotpotqa-eval400-v11-xdataset-fullr2m/summary.json`
- `r2m-api/results/gam-nqa300/summary.json`
- `r2m-api/results/r2mem-nqa300-full/summary.json`
- `memory_directions/results/narrativeqa300-v11-xdataset-fullr2m/summary.json`